

CHAPTER 4 MINING FREQUENT PATTERNS AND ASSOCIATION

- 4.1 Introduction
- 4.2 Market Basket Analysis
- 4.3 Association Rules
- 4.4 Frequent Itemset Mining Methods
- 4.5 Mining Various Kinds of Association Rules
 - 4.5.1 Multilevel Association Rules
 - 4.5.2 Multidimensional Association Rules
 - 4.5.3 Quantitative Association Rules
- 4.6 Association Rule Mining to Correlation Analysis
- 4.7 Review Questions

4.1 Introduction

Frequent patterns are patterns (such as itemsets, subsequences or substructures) that appear in a data set frequently. For example, a set of items, such as *milk and bread* that appear frequently together in a transaction data set is a *frequent itemset*. A subsequence, such as buying first a PC, then a digital camera, and then a memory card, if it occurs

frequently in a shopping history database is a (*frequent*) *sequential pattern*. Finding such frequent patterns plays an essential role in mining associations, correlations and many other interesting relationships among data. Moreover, it helps in data classification, clustering and other data mining tasks as well. Thus frequent pattern mining has become an important data mining task and a focused theme in data mining research.

4.2 Market Basket Analysis

Frequent itemset mining leads to the discovery of associations and correlations among items in large transactional or relational data sets. With massive amounts of data continuously being collected and stored, many industries are becoming interested in mining such patterns from their databases. The discovery of interesting correlation relationships among huge amounts of business transaction records can help in many business decision-making processes, such as catalog design, cross-marketing and customer shopping behavior analysis. A typical example of *frequent itemset mining* is *market basket analysis*. This process analyzes customer buying habits by finding associations between the different items that customers place in their "shopping baskets". The discovery of such associations can help retailers develop marketing strategies by gaining insight into which items are frequently purchased together by customers. For instance, if customers are buying milk, how likely are they to also buy bread (and what kind of bread) on the same trip to the supermarket? Such information can lead to increased sales by helping retailers do selective marketing and plan their shelf space.

4.3 Association Rules

If we think of the universe as the set of items available at the store, then each item has a boolean variable representing the presence or

absence of that item. Each basket can then be represented by a boolean vector of values assigned to these variables. The boolean vectors can be analyzed for buying patterns that reflect items that are frequently associated or purchased together. These patterns can be represented in the form of association rules. For example, the information that customers who purchase computers also tend to buy antivirus software at the same time is represented in association rule as given below.

Computer $\Rightarrow$ *antivirus software* [support=2%, confidence=60%]

(Rule support and confidence are two measures of rule interestingness. They respectively reflect the usefulness and certainty of discovered rules.) A support of 2% for association rule means that 2% of all the transactions under analysis show that computer and antivirus software are purchased together. A confidence of 60% means that 60% of the customers who purchased a computer also bought the software. Typically, association rules are considered interesting if they satisfy both a minimum support threshold and a minimum confidence threshold. Such thresholds can be set by users or domain experts. Additional analysis can be performed to uncover interesting statistical correlations between associated items.

4.4 Frequent Itemset Mining Methods

In this section, we will learn methods for mining the simplest form of frequent patterns which is single-dimensional, single-level, boolean frequent itemsets such as those discussed for market basket analysis.

A set of items is referred to as an *itemset*. An itemset that contains k items is a k -*itemset*. The set {computer, antivirus software} is a 2-itemset. The occurrence frequency of an itemset is the number of transactions that contain the itemset. This is also known, simply, as the

frequency, support count, or count of the itemset. If the relative support of an itemset I satisfies a pre-specified minimum support threshold (i.e., the absolute support of I satisfies the corresponding minimum support count threshold), then I is a frequent itemset. The set of frequent k -itemsets is commonly denoted by L_k . Rules that satisfy both a minimum support threshold (min_sup) and a minimum confidence threshold (min_conf) are called strong. By convention, we write support and confidence values so as to occur between 0% and 100%, rather than 0 to 1.0. In general, association rule mining can be viewed as a two-step process.

1. *Find all frequent itemsets*: By definition, each of these itemsets will occur at least as frequently as a predetermined minimum support count, min_sup.
2. *Generate strong association rules from the frequent itemsets*: By definition, these rules must satisfy minimum support and minimum confidence.

Apriori is an algorithm proposed by R. Agrawal and R. Srikant in 1994 for mining frequent itemsets for boolean association rules. The name of the algorithm is based on the fact that the algorithm uses prior knowledge of frequent itemset properties. The key concepts in apriori algorithm are as follows.

- **Frequent itemsets**: The sets of item which has minimum support (denoted by L_i for i^{th} -itemset).
- **Apriori property**: Any subset of frequent itemset must also be frequent.
- **Join operation**: To find L_k , a set of candidate k -itemsets is generated by joining L_{k-1} with itself.

Following are the important steps in an Apriori algorithm.

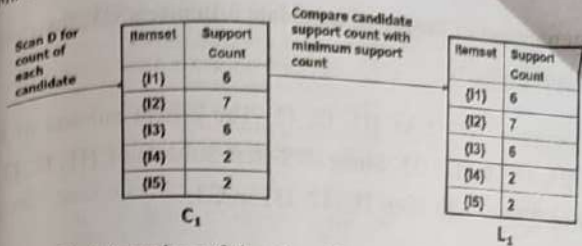
1. Find the frequent itemsets: the sets of items that have minimum support.
2. A subset of a frequent itemset must also be a frequent itemset. i.e., if $\{AB\}$ is a frequent itemset, both $\{A\}$ and $\{B\}$ should be a frequent itemset.
3. Iteratively find frequent itemsets with cardinality from 1 to k (k -itemset).
4. Use the frequent itemsets to generate association rules.

Apriori algorithm can be illustrated with an example. Consider a database D as shown in Fig 4.1 consisting of 9 transactions. Each transaction is represented by an *itemset*. Suppose minimum support required is 2 (2 out of 9 = $2/9 = 22\%$) and minimum confidence required is 70%. We have to first find out the frequent itemset using Apriori algorithm. Then, association rules will be generated using minimum support and minimum confidence.

TID	List of Items
T100	I1, I2, I5
T200	I2, I4
T300	I2, I3
T400	I1, I2, I4
T500	I1, I3
T600	I2, I3
T700	I1, I3
T800	I1, I2, I3, I5
T900	I1, I2, I3

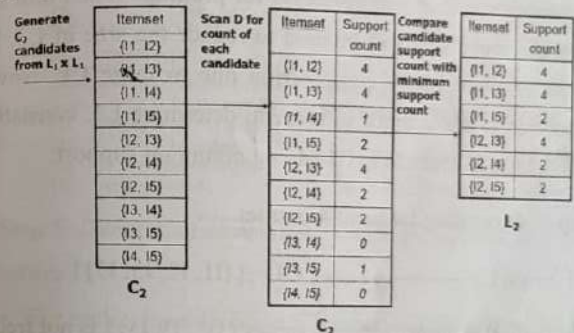
Fig. 4.1 Database D

Step 1: Generating candidate and frequent 1-itemsets with minimum support = 2.



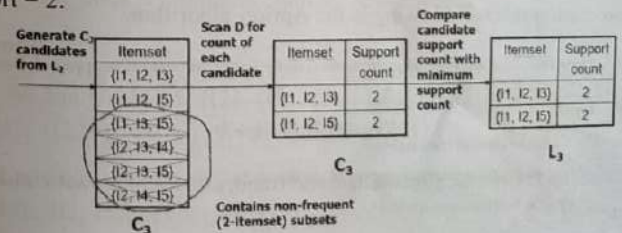
In the first iteration of the algorithm, each item is a member of the set of candidates C_k along with its support count. The set of frequent 1-itemsets L_1 , consists of the candidate 1-itemsets satisfying minimum support.

Step 2: Generating candidate and frequent 2-itemsets with minimum support = 2.



We haven't used apriori property yet.

Step 3: Generating candidate and frequent 3-itemsets with minimum support = 2.



The generation of the set of candidate 3-itemsets C_3 , involves use of the Apriori property.

For example, let's take $\{I_1, I_2, I_3\}$. The 2-item subsets of it are $\{I_1, I_2\}$, $\{I_1, I_3\}$ & $\{I_2, I_3\}$. Since all 2-item subsets of $\{I_1, I_2, I_3\}$ are members of L_2 , we will keep $\{I_1, I_2, I_3\}$ in C_3 .

Let's take another example of $\{I_2, I_3, I_5\}$ which shows how the pruning is performed. The 2-item subsets are $\{I_2, I_3\}$, $\{I_2, I_5\}$ & $\{I_3, I_5\}$. But, $\{I_3, I_5\}$ is not a member of L_2 and hence it is not frequent violating Apriori property. Thus we will have to remove $\{I_2, I_3, I_5\}$ from C_3 . Therefore, $C_3 = \{\{I_1, I_2, I_3\}, \{I_1, I_2, I_5\}\}$ after checking for all members of result of join operation for pruning. When join step is complete, the prune step will be used to reduce the size of C_3 . Prune step helps to avoid heavy computation due to large C_k . Now, the transactions in D are scanned in order to determine L_3 , consisting of those candidates 3-itemsets in C_3 having minimum support.

Step 4: Generating frequent 4-itemset.

- $L_3 \text{ Join } L_3 \longrightarrow C_4 = \{\{I_1, I_2, I_3, I_5\}\}$

This itemset is pruned since its subset $\{I_2, I_3, I_5\}$ is not frequent. Thus, $C_4 = \emptyset$, and the algorithm terminates, having found all of the frequent items. These frequent itemsets will be used to generate strong association rules. Following is the Apriori algorithm.

Algorithm: Apriori. Find frequent itemsets using an iterative level-wise approach based on candidate generation.

Input:

D , a database of transactions;

min_sup , the minimum support count threshold.

Output: L , frequent itemsets in D .

Method:

```
(1)  $L_1 = \text{find\_frequent\_1-itemsets}(D)$ ;
(2) for  $(k = 2; L_{k-1} \neq \emptyset; k++)$  {
(3)    $C_k = \text{apriori\_gen}(L_{k-1})$ ;
(4)   for each transaction  $t \in D$  // scan  $D$  for counts
(5)      $C_t = \text{subset}(C_k, t)$ ; // get the subsets of  $t$  that are candidates
(6)     for each candidate  $c \in C_t$ 
(7)        $c.\text{count}++$ ;
(8)   }
(9)    $L_k = \{c \in C_k | c.\text{count} \geq \text{min\_sup}\}$ 
(10) }
(11) return  $L = \cup_k L_k$ ;
```

procedure apriori_gen(L_{k-1} : frequent $(k-1)$ -itemsets)

```
(1) for each itemset  $l_1 \in L_{k-1}$ 
(2)   for each itemset  $l_2 \in L_{k-1}$ 
(3)     if  $(l_1[1] = l_2[1]) \wedge (l_1[2] = l_2[2])$ 
(4)        $\wedge \dots \wedge (l_1[k-2] = l_2[k-2]) \wedge (l_1[k-1] < l_2[k-1])$  then {
(5)          $c = l_1 \cup l_2$ ; // join step: generate candidates
(6)         if has_infrequent_subset( $c, L_{k-1}$ ) then
(7)           delete  $c$ ; // prune step: remove unfruitful candidate
(8)         else add  $c$  to  $C_k$ ;
(9)       }
```

procedure has_infrequent_subset(c : candidate k -itemset;

L_{k-1} : frequent $(k-1)$ -itemsets); // use prior knowledge

```
(1) for each  $(k-1)$ -subset  $s$  of  $c$ 
(2)   if  $s \notin L_{k-1}$  then
(3)     return TRUE;
(4) return FALSE;
```

Step 5: Generating association rules from frequent itemsets.

Procedure

- For each frequent itemset "I", generate all nonempty subsets of I.
- For every nonempty subset s of I, output the rule " $s \longrightarrow (I-s)$ " if $\text{support_count}(I) / \text{support_count}(s) \geq \text{min_conf}$ where min_conf is minimum confidence threshold.

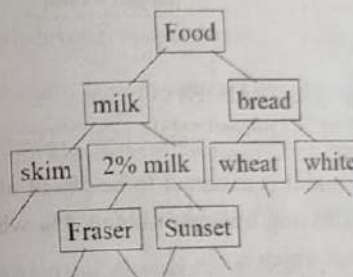
Example

We had $L = \{\{I_1\}, \{I_2\}, \{I_3\}, \{I_4\}, \{I_5\}, \{I_1, I_2\}, \{I_1, I_3\}, \{I_1, I_5\}, \{I_2, I_3\}, \{I_2, I_4\}, \{I_2, I_5\}, \{I_1, I_2, I_3\}, \{I_1, I_2, I_5\}\}$.

Let's take $I = \{I_1, I_2, I_5\}$. Its all nonempty subsets are $\{I_1, I_2\}$, $\{I_1, I_5\}$, $\{I_2, I_5\}$, $\{I_1\}$, $\{I_2\}$, $\{I_5\}$.

4.5.1 Multilevel Association Rules

Items often form hierarchies. Items at the lower level are expected to have lower support. Rules regarding itemsets at appropriate levels could be quite useful. Transaction database can be encoded based on dimensions and levels. We can explore shared multi-level mining. Consider the following concept hierarchy in a transactional database of a super market.



Mining multi-level associations can be of the following.

- A top_down, progressive deepening approach:

First find high-level strong rules:

milk $\rightarrow$ bread [20%, 60%].

Then find their lower-level "weaker" rules:

2% milk $\rightarrow$ wheat bread [6%, 50%].

- Variations at mining multiple-level association rules.

Level-crossed association rules:

2% milk $\rightarrow$ Wonder wheat bread

Association rules with multiple, alternative hierarchies:

2% milk $\rightarrow$ Wonder bread

Let minimum confidence threshold is 70%. The resulting association rules are shown below, each listed with its confidence.

- R1: $I1 \wedge I2 \rightarrow I5$. Confidence = $\text{support_count}\{I1, I2, I5\} / \text{support_count}\{I1, I2\} = 2/4 = 50\%$. Hence R1 is **rejected**.
- R2: $I1 \wedge I5 \rightarrow I2$. Confidence = $\text{support_count}\{I1, I2, I5\} / \text{support_count}\{I1, I5\} = 2/2 = 100\%$. Hence R2 is **selected**.
- R3: $I2 \wedge I5 \rightarrow I1$. Confidence = $\text{support_count}\{I1, I2, I5\} / \text{support_count}\{I2, I5\} = 2/2 = 100\%$. Hence R3 is **selected**.
- R4: $I1 \rightarrow I2 \wedge I5$. Confidence = $\text{support_count}\{I1, I2, I5\} / \text{support_count}\{I1\} = 2/6 = 33\%$. Hence R4 is **rejected**.
- R5: $I2 \rightarrow I1 \wedge I5$. Confidence = $\text{support_count}\{I1, I2, I5\} / \text{support_count}\{I2\} = 2/7 = 29\%$. Hence R5 is **rejected**.
- R6: $I5 \rightarrow I1 \wedge I2$. Confidence = $\text{support_count}\{I1, I2, I5\} / \text{support_count}\{I5\} = 2/2 = 100\%$. Hence R6 is **selected**.

In this way, we have found three strong association rules.

4.5 Mining Various Kinds of Association Rules

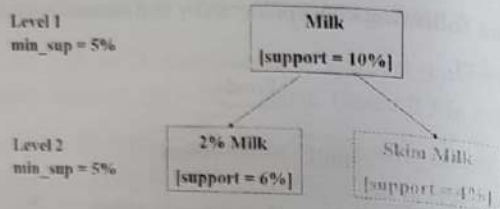
Mining various kinds of association rules in transactional and/or relational databases and data warehouses include the following.

- Multilevel association rules
- Multidimensional association rules
- Quantitative association rules

Multilevel association rules involve concepts at different levels of abstraction. Multidimensional association rules involve more than one dimension or predicate (e.g. rules relating what a customer buys as well as the customer's age.) Quantitative association rules involve numeric attributes that have an implicit ordering among values (e.g. age).

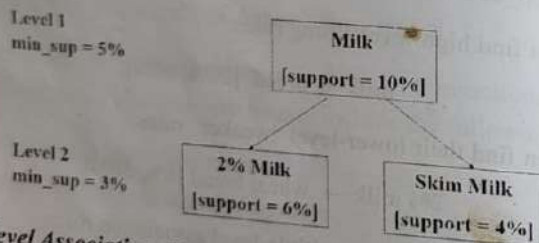
Multi-level Association: Uniform Support vs. Reduced Support

Uniform Support: This means the same minimum support for all levels. There will be only one minimum support threshold. In this there is no need to examine itemsets containing any item whose ancestors do not have minimum support. Following shows an example for uniform support.



Here minimum support is assumed to be 5% at all levels. We can see that Skim milk is having a support count 4% which is less than minimum support count which is 5%.

Reduced Support: This means that the minimum support will be a reduced minimum support at lower levels. Following shows an example for reduced support.



Multi-level Association: Redundancy Filtering

Some rules may be redundant due to "ancestor" relationships between items. Consider the following example.

milk $\rightarrow$ wheat bread [support = 8%, confidence = 70%]

2% milk $\rightarrow$ wheat bread [support = 2%, confidence = 72%]

We say the first rule is an ancestor of the second rule. A rule is redundant if its support is close to the "expected" value, based on the rule's ancestor.

Multi-Level Mining: Progressive Deepening

A top-down, progressive deepening approach can be considered as follows.

First mine high-level frequent items:

milk (15%), bread (10%)

Then mine their lower-level "weaker" frequent itemsets:

2% milk (5%), wheat bread (4%)

4.5.2 Multidimensional Association Rules

Single dimensional association rule

Consider the following association rule.

$\text{Buys}(X, \text{"milk"}) \rightarrow \text{Buys}(X, \text{"bread"})$

We can refer to an association rule as a single dimensional or intra dimensional association rule when it contains a single distinct predicate (e.g. *Buys*) with multiple occurrences (i.e., the predicate occurs more than once within the rule).

Multi-dimensional association rule

Association rules that involve two or more dimensions or predicates can be referred to as multidimensional association rules. Consider the following association rule.

$\text{Age}(X, \text{"20...29"}) \wedge \text{Occupation}(X, \text{"student"}) \Rightarrow \text{Buys}(X, \text{"laptop"})$

The above rule contains three predicates (age, occupation and buys), each of which occurs only once in the rule. Hence we say that it has no repeated predicates. Multidimensional association rules with no repeated predicates are called *inter-dimensional association rules*. We can also mine multidimensional association rules with repeated predicates, which contain multiple occurrences of some predicates. These rules are called *hybrid-dimensional association rules*. An example of such a rule is the following, where the predicate *Buys* is repeated.

$$\text{Age}(X, "20...29") \wedge \text{Buys}(X, "laptop") \Rightarrow \text{Buys}(X, "HP \text{ printer}")$$

The database attributes can be *categorical* or *quantitative*. *Categorical attributes* have a finite number of possible values, with no ordering among the values (e.g., occupation, brand, color). Categorical attributes are also called nominal attributes, because their values are "names of things". Following is an example for categorical association rule.

$$\text{Occupation}(X, "Doctor") \rightarrow \text{Brand}(X, "Maruti") \rightarrow \text{Color}(X, "White")$$

4.5.3 Quantitative Association Rules

Quantitative attributes are numeric and have an implicit ordering among values (e.g., age, income, price). Techniques for mining multidimensional association rules can be categorized into two basic approaches regarding the treatment of quantitative attributes. In the first approach, quantitative attributes are discretized using predefined concept hierarchies. This discretization occurs before mining. For instance, a concept hierarchy for *age* may be used to replace the original numeric values of this attribute by interval labels, such as "20...25", "26...30", "31...40", and so on.

In the second approach, quantitative attributes are discretized or clustered into "bins" based on the distribution of the data. These bins may be further combined during the mining process. The discretization process is dynamic and established so as to satisfy some mining criteria, such as maximizing the confidence of the rules mined. Because this strategy treats the numeric attribute values as quantities rather than as predefined ranges or categories, association rules mined from this approach are also referred to as (dynamic) quantitative association rules. An example of such a 2-D quantitative association rule is given below. Consider the 4 association rules given below.

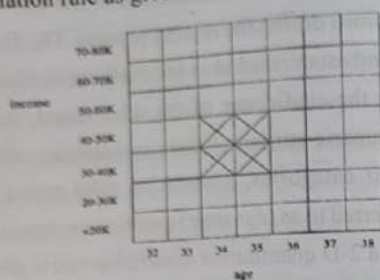
1. $\text{Age}(X, "34") \rightarrow \text{Income}(X, "31K - 40K") \rightarrow \text{Buys}(X, "high \text{ resolution TV}")$
2. $\text{Age}(X, "35") \rightarrow \text{Income}(X, "31K - 40K") \rightarrow \text{Buys}(X, "high \text{ resolution TV}")$
3. $\text{Age}(X, "34") \rightarrow \text{Income}(X, "41K - 50K") \rightarrow \text{Buys}(X, "high \text{ resolution TV}")$
4. $\text{Age}(X, "35") \rightarrow \text{Income}(X, "41K - 50K") \rightarrow \text{Buys}(X, "high \text{ resolution TV}")$

The above four association rules can be replaced using the below single association rule.

$$\text{Age}(X, "34...35") \wedge \text{Income}(X, "31K...50K") \Rightarrow \text{Buys}(X, "HDTV")$$

A grid-based technique described below assumes that the initial association rules can be clustered into rectangular regions. Before performing the clustering, smoothing techniques can be used to help remove noise and outliers from the data. Rectangular clusters may over simplify the data. Following shows an example for grid-based method.

The above 4 association rules can be mapped to a grid and arrive at a single association rule as given below.



$Age(X, "34..35") \wedge Income(X, "31K...50K") \Rightarrow Buys(X, "HDTV")$

4.6 Association Rule Mining to Correlation Analysis

Most association rule mining algorithms employ a support-confidence framework. A good number of rules so generated are still not interesting to the users. This is especially true when mining at low support thresholds or mining for long patterns. This has been one of the major bottlenecks for successful application of association rule mining. In this section, we first look at how even strong association rules can be uninteresting and misleading. We then discuss how the support-confidence framework can be supplemented with additional interestingness measures based on statistical significance and correlation analysis.

A misleading "strong" association rule can be as follows. Suppose we are interested in analyzing transactions at an electronics store given in Table 4.1 with respect to the purchase of computer games and videos. Let game refer to the transactions containing computer games and video refer to those containing videos. Of the 10,000 transactions analyzed, the data show that 6,000 of the customer transactions included computer

games, while 7,500 included videos and 4,000 included both computer games and videos. Suppose that a data mining program for discovering association rules is run on the data, using a minimum support of 30% and a minimum confidence of 60%. The following association rule is discovered.

$Buys(X, "computer\ games") \Rightarrow Buys(X, "videos")$ [support = 40%, confidence = 66%]

This is a strong association rule as it satisfies the minimum support and minimum confidence thresholds, respectively. However, this rule is misleading because the probability of purchasing videos is 75%, which is even larger than 66%. In fact, computer games and videos are negatively associated because the purchase of one of these items actually decreases the likelihood of purchasing the other. Without fully understanding this phenomenon, we could easily make unwise business decisions based on the above rule.

It does not measure the real strength (or lack of strength) of the correlation and implication between A and B. Hence, alternatives to the support-confidence framework can be useful in mining. As we have seen above, the support and confidence measures are insufficient at filtering out uninteresting association rules. To tackle this weakness, a correlation measure can be used to augment the support-confidence framework for association rules. This leads to correlation rules of the form

$$A \Rightarrow B [\text{support, confidence, correlation}]$$

That is, a correlation rule is measured not only by its support and confidence but also by the correlation between itemsets A and B. There are many different correlation measures from which to choose. *Lift* is a simple correlation measure that is given as follows.

The occurrence of itemset A is independent of the occurrence of itemset B if $P(A \cup B) = P(A)P(B)$; otherwise, itemsets A and B are dependent and correlated as events. This definition can easily be extended to more than two itemsets. The lift between the occurrence of A and B can be measured by computing

$$\text{lift}(A, B) = P(A \cup B) / P(A)P(B).$$

If the resulting value of above equation is less than 1, then the occurrence of A is negatively correlated with the occurrence of B. If the resulting value is greater than 1, then A and B are positively correlated, meaning that the occurrence of one implies the occurrence of the other. If the resulting value is equal to 1, then A and B are independent and there is no correlation between them. In other words, it assesses the degree to which the occurrence of one "lifts" the occurrence of the other.

For example, if A corresponds to the sale of computer games and B corresponds to the sale of videos, then given the current market conditions, the sale of games is said to increase or "lift" the likelihood of the sale of videos by a factor of the value returned by the value of above equation.

Correlation analysis using lift

To help filter out misleading "strong" associations of the form $A \Rightarrow B$ from the data of previous example, we need to study how the two itemsets, A and B, are correlated. Let game refer to the transactions that do not contain computer games, and video refer to those that do not contain videos. The transactions can be summarized in a contingency table, as shown in Table 4.1.

A 2×2 contingency table summarizing the transactions with respect to game and video purchases.

	game	<u>game</u>	Σ_{row}
video	4,000	3,500	7,500
<u>video</u>	2,000	500	2,500
Σ_{col}	6,000	4,000	10,000

Table 4.1 Summarization of Transactions

From the table, we can see that the probability of purchasing a computer game is $P(\{\text{game}\}) = 0.60$, the probability of purchasing a video is $P(\{\text{video}\}) = 0.75$, and the probability of purchasing both is $P(\{\text{game}, \text{video}\}) = 0.40$.

By the equation, the lift of rule is

$$P(\{\text{game}, \text{video}\}) / (P(\{\text{game}\}) \times P(\{\text{video}\})) = 0.40 / (0.60 \times 0.75) = 0.89.$$

Because this value is less than 1, there is a negative correlation between the occurrence of {game} and {video}. The numerator is the likelihood of a customer purchasing both. The interesting data relationship denominator is what the likelihood would have been if the two purchases were completely independent. Such a negative correlation cannot be identified by a support confidence framework. A rule with negative correlation can be easily wiped out as it is not useful.

4.7 Review Questions

1. What is a frequent itemset?
2. What is a frequent pattern?
3. What is a frequent sequential pattern?
4. What is an association rule? Give example.

5. What is market basket analysis?
6. Which is the algorithm used for mining frequent itemsets?
7. What is apriori property?
8. Explain the concept of apriori algorithm.
9. How do you generate association rules from frequent itemsets?
10. Explain different types of association rules.
11. What is lift?
12. What is the role of correlation analysis in association rule mining?

CLASSIFICATION & PREDICTION

- 5.1 Introduction
- 5.2 Classification
 - 5.2.1 Issues in Classification and Prediction
- 5.3 Classification by Decision Tree Induction
- 5.4 Bayesian Classification
- 5.5 Rule Based Classification
- 5.6 Support Vector Machines
- 5.7 Learning from your Neighbours
- 5.8 Prediction
- 5.9 Review Questions

5.1 Introduction

There are two forms of data analysis that can be used for extracting models describing important classes or to predict future data trends.

These two forms are as follows.

- Classification
- Prediction

Classification predicts categorical class labels. It classifies data (constructs a model) based on the training set and the values (class labels) in a classifying attribute and uses it in classifying new data. For example, we can build a classification model to categorize bank loan applications as either safe or risky.

Prediction models continuous valued functions, i.e., predicts unknown or missing values. For example a prediction model can be used to predict the expenditures in rupees of potential customers on computer equipment given their income and occupation.

Typical applications of classification and prediction include credit approval, target marketing, medical diagnosis, treatment effectiveness analysis etc.

Supervised vs. Unsupervised Learning

Supervised learning is also known as *classification*. Supervision involves the training data (observations, measurements, etc.) which are accompanied by labels indicating the class of the observations. New data is classified based on the training data set.

Unsupervised learning is also known as *clustering*. In unsupervised learning, the class labels of training data is unknown. A set of measurements or observations will be given with an aim of establishing the existence of classes or clusters in the data.

5.2 Classification

Classification is a two-step process as follows.

1. **Model construction:** This involves describing a set of predetermined classes. Each tuple or sample is assumed to belong to a predefined

class, as determined by the class label attribute. The set of tuples used for model construction is known as training data set. This step is the **learning step** or the **learning phase**. In this step the classification algorithms build the classifier. The classifier is built from the training data set made up of database tuples and their associated class labels. Each tuple that constitutes the training set is referred to as a category or class. These tuples can also be referred to as sample, object or data points. The model is represented as **classification rules**, **decision trees**, or **mathematical formulae**. Fig. 5.1 shows an example for model construction.

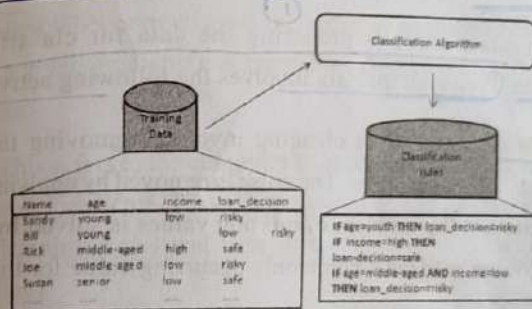


Fig. 5.1 Model Construction

2. **Model usage:** Once the model is constructed, this model is used for classifying future or unknown objects. In this step, the classifier is used for classification. Here the test data is used to estimate the accuracy of classification rules. Accuracy rate is the percentage of test set samples that are correctly classified by the model. Test set is independent of training set, otherwise over-fitting will occur. The classification rules can be applied to the new data tuples if the accuracy is considered acceptable. Fig. 5.2 shows the model usage in classification.

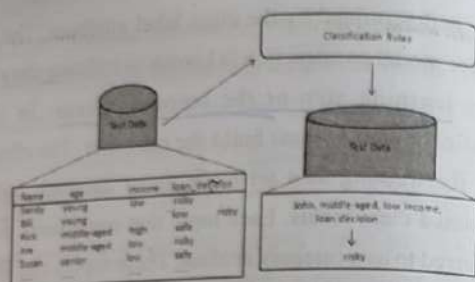


Fig. 5.2 Model Usage

5.2.1 Issues in Classification and Prediction

The major issue is preparing the data for classification and prediction. Preparing the data involves the following activities.

Data cleaning: Data cleaning involves removing the noise and treatment of missing values. The noise is removed by applying smoothing techniques and the problem of missing values is solved by replacing a missing value with most commonly occurring value for that attribute.

Relevance Analysis: Database may also have the irrelevant attributes. Correlation analysis is used to know whether any two given attributes are related.

Data Transformation and Reduction: The data can be transformed by any of the following methods.

- Normalization:** The data is transformed using normalization. Normalization involves scaling all values for given attribute in order to make them fall within a small specified range. Normalization is used when in the learning step, the neural networks or the methods involving measurements are used.

- Generalization:** The data can also be transformed by generalizing it to the higher concept. For this purpose we can use the concept hierarchies.

Data can also be reduced by some other methods such as wavelet transformation, binning, histogram analysis and clustering.

Another issue is with evaluating classification and prediction methods. Following are the criteria for comparing the methods of classification and prediction.

Accuracy: Accuracy of classifier refers to the ability of classifier to predict the class label correctly and the accuracy of the predictor refers to how well a given predictor can guess the value of predicted attribute for a new data.

Speed: This refers to the computational cost in generating and using the classifier or predictor. This is considered as the time to construct the model (training time) and time to use the model (classification/prediction time).

Robustness: It refers to the ability of classifier or predictor to make correct predictions from given noisy data.

Scalability: Scalability refers to the ability to construct the classifier or predictor efficiently for given large amount of data.

Interpretability: It refers to what extent the classifier or predictor understands.

5.3 Classification by Decision Tree Induction

A decision tree is a structure that includes a root node, branches, and leaf nodes. Each internal node denotes a test on an attribute, each

branch denotes the outcome of a test, and each leaf node holds a class label. The topmost node in the tree is the root node.

The following decision tree in Fig 5.3 is for the concept *buys_computer* that indicates whether a customer is likely to buy a computer or not. Each internal node represents a test on an attribute. Each leaf node represents a class.

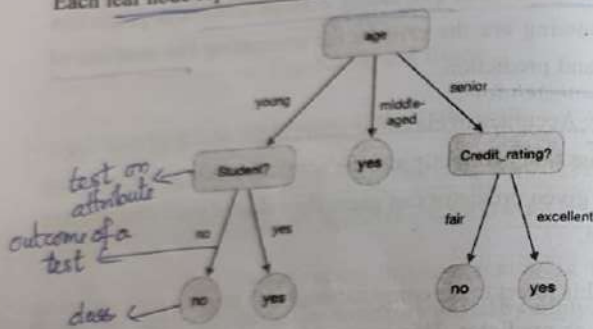


Fig. 5.3 Decision Tree for *buys_computer*

The benefits of having a decision tree are as follows.

- It does not require any domain knowledge.
- It is easy to comprehend.
- The learning and classification steps of a decision tree are simple and fast.

Decision tree generation consists of two phases.

- **Tree construction** - At start, all the training examples are at the root. Partition examples recursively based on selected attributes.
- **Tree pruning** - Identify and remove branches that reflect noise or outliers.

Use of decision tree: Classifying an unknown sample. Also test the attribute values of the sample against the decision tree.

A machine learning researcher named J. Ross Quinlan in 1980 developed a decision tree algorithm known as ID3 (Iterative Dichotomiser). Later, he presented C4.5, which was the successor of ID3. ID3 and C4.5 adopt a greedy approach. In this algorithm, there is no backtracking. The trees are constructed in a top-down recursive divide-and-conquer manner. Consider the below training data set shown in Fig 5.4. Before going for the algorithm, we need to know how attributes are selected for partitioning. The attribute selection measure can be done in one of the following ways: *information gain*, *gini index* and *gain ratio*. Here we will be discussing how to calculate *information gain* for attribute selection.

Information Gain

Let node N represent or hold the tuples of partition D. Attribute with the highest information gain is chosen as the splitting attribute for node N. Let p_i be the probability that an arbitrary tuple in D belongs to class C_i , estimated by $|C_{i,D}|/|D|$ where $|C_{i,D}|$ and $|D|$ denote the number of tuples in $C_{i,D}$ and D respectively.

Expected information (entropy) needed to classify a tuple in D:

$$\text{Info}(D) = - \sum_{i=1}^m p_i \log_2(p_i)$$

Information needed (after using A to split D into v partitions) to classify D:

$$\text{Info}_A(D) = \sum_{j=1}^v \frac{|D_j|}{|D|} \times \text{Info}(D_j)$$

Information gained by branching on attribute A:

$$\text{Gain}(A) = \text{Info}(D) - \text{Info}_A(D)$$

age	income	student	credit_rating	buys_computer
youth	high	no	fair	no
youth	high	no	excellent	no
middle-aged	high	no	fair	yes
senior	medium	no	fair	yes
senior	low	yes	fair	yes
senior	low	yes	excellent	no
middle-aged	low	yes	excellent	yes
youth	medium	no	fair	no
youth	low	yes	fair	yes
senior	medium	yes	fair	yes
youth	medium	yes	excellent	yes
middle-aged	medium	no	excellent	yes
middle-aged	high	yes	fair	yes
senior	medium	no	excellent	no

Fig. 5.4 Training Data Set

In the above training data set, there are two distinct classes $m=2$. Let $C1$ corresponds to "yes" and $C2$ corresponds to "no". $\text{Info}(D)$ can be calculated as follows.

$$\text{Info}(D) = -9/14 \log_2(9/14) - 5/14 \log_2(5/14) = 0.94 \text{ bits.}$$

$$\text{Info}_{\text{age}}(D) = 5/14 * (-2/5 \log_2(2/5) - 3/5 \log_2(3/5)) + 4/14 * (-4/4 \log_2(4/4) + 5/14 * (-3/5 \log_2(3/5) - 2/5 \log_2(2/5))) = 0.694 \text{ bits.}$$

$$\text{Gain}(\text{age}) = \text{Info}(D) - \text{Info}_{\text{age}}(D) = 0.940 - 0.694 = 0.246 \text{ bits.}$$

Similarly $\text{Gain}(\text{income})$, $\text{Gain}(\text{student})$ and $\text{Gain}(\text{credit_rating})$ are calculated in the same manner and we got the result as $\text{Gain}(\text{income}) = 0.029$ bits, $\text{Gain}(\text{student}) = 0.151$ bits, $\text{Gain}(\text{credit_rating}) = 0.048$ bits. Since $\text{Gain}(\text{age})$ is having the highest value, age is chosen as the root node for partitioning. In a similar way, as algorithm progresses, the information gain of the rest of the attributes are calculated based on which the splitting will be done.

Explanation of ID3 algorithm

Basic algorithm (a greedy algorithm)

- Tree is constructed in a top-down recursive divide-and-conquer manner.

- At start, all the training examples are at the root.
- Attributes are categorical (if continuous-valued, they are discretized in advance).
- Examples are partitioned recursively based on selected attributes.
- Test attributes are selected on the basis of a heuristic or statistical measure (e.g., information gain).

Conditions for stopping partitioning

- All samples for a given node belong to the same class.
- There are no remaining attributes for further partitioning – majority voting is employed for classifying the leaf.
- There are no samples left.

Following shows the algorithm for decision tree induction.

Algorithm: Generate decision tree. Generate a decision tree from the training tuples of data partition, D .

Input:

- Data partition, D , which is a set of training tuples and their associated class labels;
- attribute_list, the set of candidate attributes;
- Attribute selection method, a procedure to determine the splitting criterion that "best" partitions the data tuples into individual classes. This criterion consists of a splitting_attribute and, possibly, either a split-point or splitting_subset.

Output: A decision tree.

Method:

- create a node N ;
- if tuples in D are all of the same class, C , then
- return N as a leaf node labeled with the class C ;
- if attribute_list is empty then
- return N as a leaf node labeled with the majority class in D ; // majority voting
- apply Attribute selection method(D , attribute_list) to find the "best" splitting criterion;
- label node N with splitting_attribute;
- if splitting_attribute is discrete-valued and multiway splits allowed then // not restricted to binary trees
- attribute_list ← attribute_list – splitting_attribute; // remove splitting_attribute
- for each outcome j of splitting_attribute
- // partition the tuples and grow subtrees for each partition
- let D_j be the set of data tuples in D satisfying outcome j ; // a partition
- if D_j is empty then
- attach a leaf labeled with the majority class in D to node N ;
- else attach the node returned by Generate_decision_tree(D_j , attribute_list) to node N ;
- endfor
- return N ;

The result of above ID3 algorithm will be a decision tree as shown in Fig. 5.3.

Tree pruning is performed in order to remove anomalies in the training data due to noise or outliers. The pruned trees are smaller and less complex. There are two approaches to prune a tree.

- ✓ **Pre-pruning** – The tree is pruned by halting its construction early.
- ✓ **Post-pruning** – This approach removes a sub-tree from a fully grown tree.

The cost complexity is measured by two parameters namely the number of leaves in the tree and error rate of the tree.

5.4 Bayesian Classification

Bayesian classification is based on Bayes' Theorem. Bayesian classifiers are the statistical classifiers. Bayesian classifiers can predict class membership probabilities such as the probability that a given tuple belongs to a particular class. Naïve Bayesian classifier, is a simple Bayesian classifier which has comparable performance with decision tree and selected neural network classifiers.

Naïve Bayesian classifier is incremental in nature. Each training example can incrementally increase or decrease the probability that a hypothesis is correct. This means prior knowledge can be combined with observed data.

Bayes' Theorem

Bayes' Theorem is named after Thomas Bayes. There are two types of probabilities.

- Posterior Probability $[P(H/X)]$
- Prior Probability $[P(H)]$

where X is data tuple and H is some hypothesis.

- ✓ Let X be a data sample ("evidence"): class label is unknown.
 - ✓ Let H be a hypothesis that X belongs to class C .
 - ✓ Classification is to determine $P(H/X)$, the probability that the hypothesis holds given the observed data sample X .
 - ✓ $P(H)$ (prior probability), the initial probability. E.g., X will buy computer, regardless of age, income etc.
 - ✓ $P(X)$: probability that sample data is observed.
 - ✓ $P(X|H)$ (posteriori probability), the probability of observing the sample X , given that the hypothesis holds. E.g., Given that the probability of X will buy computer, that X is 31. 40 age group and medium income.
 - ✓ Given training data X , posteriori probability of a hypothesis H , $P(H/X)$, follows the Bayes theorem.
- $$P(H|X) = \frac{P(X|H)P(H)}{P(X)}$$
- ✓ Informally, this can be written as posteriori = likelihood x prior / evidence.
 - ✓ Predicts X belongs to C_2 iff the probability $P(C_i|X)$ is the highest among all the $P(C_k|X)$ for all the k classes.

Naïve Bayesian Classifier

- ✓ Let D be a training set of tuples and their associated class labels, and each tuple is represented by an n -D attribute vector $X = (x_1, x_2, \dots, x_n)$.
- ✓ Suppose there are m classes $C_1, C_2, \dots, C_m$.
- ✓ Classification is to derive the maximum posteriori, i.e., the maximal $P(C_i|X)$.

- ✓ This can be derived from Bayes' theorem.

$$P(C_i | X) = \frac{P(X | C_i) P(C_i)}{P(X)}$$

- ✓ Since $P(X)$ is constant for all classes, only

$P(C_i | X) = P(X | C_i) P(C_i)$ needs to be maximized.

- ✓ A simplified assumption: attributes are conditionally independent (i.e., no dependence relation between attributes):

$$P(X | C_i) = \prod_{k=1}^n P(x_k | C_i) = P(x_1 | C_i) \times P(x_2 | C_i) \times \dots \times P(x_n | C_i)$$

- ✓ This greatly reduces the computation cost: Only counts the class distribution.

- ✓ If A_k is categorical, $P(x_k | C_i)$ is the # of tuples in C_i having value x_k for A_k divided by $|C_i|$ (# of tuples of C_i in D)

- ✓ If A_k is continuous-valued, $P(x_k | C_i)$ is usually computed based on Gaussian distribution with a mean μ and standard deviation σ

$$g(x, \mu, \sigma) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(x-\mu)^2}{2\sigma^2}} \text{ and } P(x_k | C_i) \text{ is } P(X | C_i) = g(x_k, \mu_{C_i}, \sigma_{C_i})$$

Example

Consider the training data set given in Fig. 5.4. There are two classes in this training data set. Class: C_1 : buys_computer = 'yes'

C_2 : buys_computer = 'no'

We need to classify the following data sample to either one of the above classes C_1 or C_2 .

$X = (\text{age} = \text{youth}, \text{income} = \text{medium}, \text{student} = \text{yes}, \text{credit_rating} = \text{fair})$

$$P(C_1): P(\text{buys_computer} = \text{"yes"}) = 9/14 = 0.643$$

$$P(\text{buys_computer} = \text{"no"}) = 5/14 = 0.357$$

- ✓ Compute $P(X | C_i)$ for each class

$$P(\text{age} = \text{"youth"} | \text{buys_computer} = \text{"yes"}) = 2/9 = 0.222$$

$$P(\text{age} = \text{"youth"} | \text{buys_computer} = \text{"no"}) = 3/5 = 0.6$$

$$P(\text{income} = \text{"medium"} | \text{buys_computer} = \text{"yes"}) = 4/9 = 0.444$$

$$P(\text{income} = \text{"medium"} | \text{buys_computer} = \text{"no"}) = 2/5 = 0.4$$

$$P(\text{student} = \text{"yes"} | \text{buys_computer} = \text{"yes"}) = 6/9 = 0.667$$

$$P(\text{student} = \text{"yes"} | \text{buys_computer} = \text{"no"}) = 1/5 = 0.2$$

$$P(\text{credit_rating} = \text{"fair"} | \text{buys_computer} = \text{"yes"}) = 6/9 = 0.667$$

$$P(\text{credit_rating} = \text{"fair"} | \text{buys_computer} = \text{"no"}) = 2/5 = 0.4$$

$$P(X | C_1): P(X | \text{buys_computer} = \text{"yes"}) = 0.222 \times 0.444 \times 0.667 \times 0.667 = 0.044$$

$$P(X | \text{buys_computer} = \text{"no"}) = 0.6 \times 0.4 \times 0.2 \times 0.4 = 0.019$$

$$P(X | C_i) * P(C_i): P(X | \text{buys_computer} = \text{"yes"}) * P(\text{buys_computer} = \text{"yes"}) = 0.044 \times 0.643 = 0.028$$

$$P(X | \text{buys_computer} = \text{"no"}) * P(\text{buys_computer} = \text{"no"}) = 0.019 \times 0.357 = 0.007$$

Since the maximum value is for $P(\text{buys_computer} = \text{"yes"})$,

X belongs to class ("buys_computer = yes").

5.5 Rule Based Classification

Rule-based classifier makes use of a set of IF-THEN rules for classification. We can express a rule in the following form.

IF condition THEN conclusion

Let us consider a rule R_1 : IF age = youth AND student = yes THEN buys_computer = yes

The IF part of the rule is called rule antecedent or precondition. The THEN part of the rule is called rule consequent. The antecedent

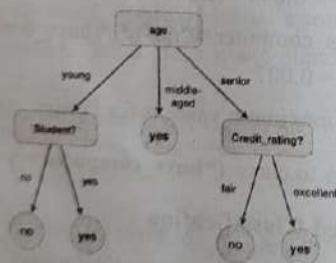
part consists of one or more attribute tests and these tests are connected using logical AND. The consequent part consists of class prediction. We can also write rule R1 as follows.

$R1: ((age = young) \wedge (student = yes)) (buys_computer = yes)$

If the condition holds true for a given tuple, then the antecedent is satisfied.

Rule Extraction

Here we will learn how to build a rule-based classifier by extracting IF-THEN rules from a decision tree. Rules are easier to understand than large trees. One rule is created for each path from the root to a leaf. The leaf node holds the class prediction, forming the rule consequent. Rules are mutually exclusive and exhaustive. Consider the following decision tree for *buys_computer*.



Following are the rules extracted from *buys_computer* decision tree.

IF *age* = young AND *student* = no THEN *buys_computer* = no

IF *age* = young AND *student* = yes THEN *buys_computer* = yes

IF *age* = middle-aged THEN *buys_computer* = yes

IF *age* = senior AND *credit_rating* = excellent THEN *buys_computer* = yes

IF *age* = senior AND *credit_rating* = fair THEN *buys_computer* = no

Assessment of a rule is done by coverage and accuracy. The number of rules covered by a rule R is denoted by

$$n_{covers} = \# \text{ of tuples covered by } R$$

The number of rules correctly classified by rule R is given by

$$n_{correct} = \# \text{ of tuples correctly classified by } R$$

The coverage of rule R is given by

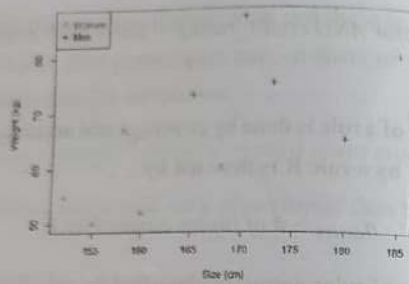
$$coverage(R) = n_{covers} / |D| \quad /* D \text{ is the total number of tuples in training data set } */$$

The accuracy of rule R is given by number of tuples correctly classified by R divided by number of tuples correctly covered by R.

$$accuracy(R) = n_{correct} / n_{covers}$$

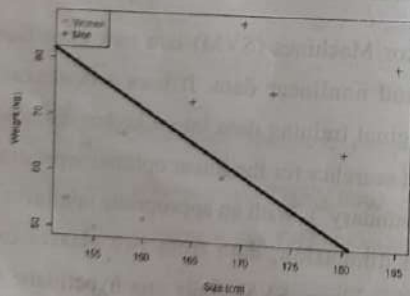
5.6 Support Vector Machines

Support Vector Machines (SVM) is a new classification method for both linear and nonlinear data. It uses a nonlinear mapping to transform the original training data into a higher dimension. With the new dimension, it searches for the linear optimal separating hyperplane (i.e., "decision boundary"). With an appropriate nonlinear mapping to a sufficiently high dimension, data from two classes can always be separated by a hyperplane. SVM finds this hyperplane using support vectors ("essential" training tuples) and margins (defined by the support vectors). Consider the following training data set.



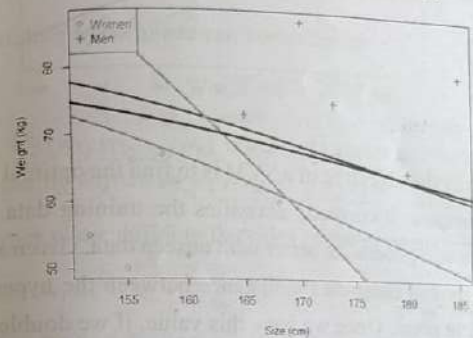
We have plotted the size and weight of several people, and there is also a way to distinguish between men and women (+ for men and o for women). With such data, using a SVM will allow us to answer the following question: Given a particular data point (weight and size), is the person a man or a woman? For instance: if someone measures 175 cm and weights 80 kg, is it a man or a woman?

Just by looking at the plot, we can see that it is possible to separate the data. For instance, we could trace a line and then all the data points representing men will be above the line, and all the data points representing women will be below the line. Such a line is called a **separating hyperplane** and is depicted below.

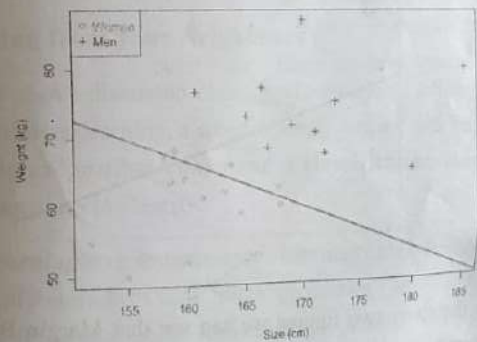


A hyperplane is a generalization of a plane. In one dimension, a hyperplane is called a point. In two dimensions it is a line, in three

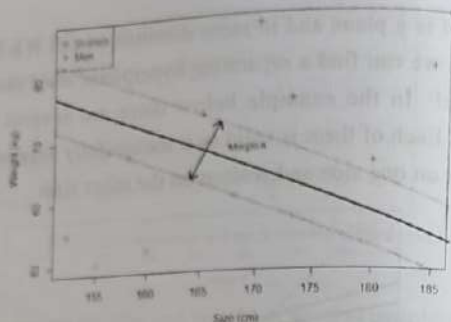
dimensions it is a plane and in more dimensions call it a hyperplane. The fact that we can find a separating hyperplane, does not mean it is the best one! In the example below there are several separating hyperplanes. Each of them is valid as it successfully separates our data set with men on one side and women on the other side.



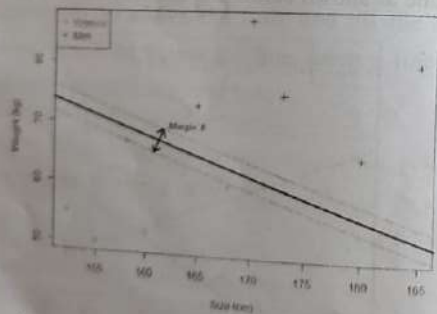
There can be a lot of separating hyperplanes. Suppose we select one hyperplane as shown below and use it to classify on real life data.



This time, it makes some mistakes as it wrongly classifies three women. So we will try to select a hyperplane as far as possible from data points from each category as follows.



That's why the objective of a SVM is to find the optimal separating hyperplane because it correctly classifies the training data. Also it is the one which will generalize better with unseen data. Given a particular hyperplane, we can compute the distance between the hyperplane and the closest data point. Once we have this value, if we double it we will get what is called the margin. For another hyperplane, the margin will look like as follows.



From the above two figures we can see that Margin B is smaller than Margin A. We can make the following observations:

- If a hyperplane is very close to a data point, its margin will be small.

- The farther a hyperplane is from a data point, the larger its margin will be. This means that the optimal hyperplane will be the one with the biggest margin.

That is why the objective of the SVM is to find the optimal separating hyperplane which maximizes the margin of the training data.

A separating hyper plane can be written as

$$W \cdot X + b = 0$$

where $W = \{w_1, w_2, \dots, w_n\}$ is a weight vector and b a scalar (bias).

For 2-D it can be written as $w_0 + w_1 x_1 + w_2 x_2 = 0$

The hyper plane defining the sides of the margin:

$$H1: w_0 + w_1 x_1 + w_2 x_2 \geq 1 \quad \text{for } y_i = +1, \text{ and}$$

$$H2: w_0 + w_1 x_1 + w_2 x_2 \leq -1 \quad \text{for } y_i = -1$$

Any training tuples that fall on hyper planes $H1$ or $H2$ (i.e., the sides defining the margin) are called **support vectors**.

5.7 Learning from your Neighbours

All the above discussed classification methods belongs to **eager learning**. In eager learning, a set of training data set will be given. The classification algorithm constructs a classification model before receiving new data to classify.

Lazy learning is an instance-based learning. In this learning, training data is simply stored (or only minor processing) and waits until it is given a test tuple to classify. K-Nearest Neighbor (KNN) algorithm is an example of instance-based learning.

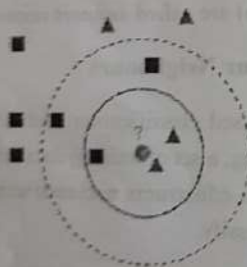
In KNN, all instances correspond to points in the n -D space. The nearest neighbor are defined in terms of Euclidean distance, $\text{dist}(X1,$

X2). K-nearest neighbor classifier searches the pattern space for the k training tuples that are closest to the unknown tuple. For categorical attributes, if 2 tuples are identical, the difference is taken as 0 otherwise the difference is 1.

KNN Algorithm

1. Calculate " $d(x, x_i)$ " $i = 1, 2, \dots, n$; where d denotes the Euclidean distance between the points.
2. Arrange the calculated n Euclidean distances in non-decreasing order.
3. Let k be a +ve integer, take the first k distances from this sorted list.
4. Find those k -points corresponding to these k -distances.
5. Let k_i denotes the number of points belonging to the i^{th} class among k points i.e. $k_i \geq 0$.
6. If $k_i > k_j, \forall i \neq j$, then put x in class i .

Example



Consider the above given figure. The test sample (dot) should be classified either to squares or to triangles. If $k = 3$ (solid line circle) it is assigned to the triangles because there are 2 triangles and only 1 square inside the inner circle. If $k = 5$ (dashed line circle) it is assigned to the squares (3 squares vs. 2 triangles inside the outer circle).

Choosing the right value for K

To select the K that's right for your data, we run the KNN algorithm several times with different values of K and choose the K that reduces the number of errors we encounter while maintaining the algorithm's ability to accurately make predictions when it's given data it hasn't seen before. Here are some things to keep in mind.

- As we decrease the value of K to 1, our predictions become less stable.
- Inversely, as we increase the value of K , our predictions become more stable due to majority voting / averaging, and thus, more likely to make more accurate predictions (up to a certain point). Eventually, we begin to witness an increasing number of errors. It is at this point we know we have pushed the value of K too far.
- In cases where we are taking a majority vote (e.g. picking the mode in a classification problem) among labels, we usually make K an odd number to have a tiebreaker.

Advantages

- ✓ The algorithm is simple and easy to implement.
- ✓ There's no need to build a model, tune several parameters, or make additional assumptions.
- ✓ The algorithm is versatile. It can be used for classification, regression and search.

Disadvantages

- The algorithm gets significantly slower as the number of examples and/or predictors/independent variables increase.

5.8 Prediction

(Numerical) prediction is similar to classification. Prediction is also a two-step process.

- ✓ Construct a model using training data set.
- ✓ Use this model to predict continuous or ordered value for a given input.

Prediction is different from classification. Classification refers to predict categorical class label while prediction models continuous-valued functions. Major method for prediction is **regression**. It models the relationship between one or more independent or **predictor** variables and a dependent or **response** variable. Regression analysis involves the following types.

- ✓ Linear regression
- ✓ Multiple linear regression
- ✓ Non-linear regression

Linear Regression

Linear regression involves a response variable y and a single predictor variable x .

$$y = w_0 + w_1 x$$

where w_0 (y-intercept) and w_1 (slope) are regression coefficients. These coefficients can be solved for by the method of least squares, which estimates the best-fitting straight line as the one that minimizes the error between the actual data and the estimate of the line. Let D be a training set consisting of values of predictor variable, x , for some population and their associated values for response variable, y . The training set contains $|D|$ data points of the form $(x_1, y_1), (x_2, y_2), \dots$

The regression coefficients can be estimated using this method with the following equations.

$$w_1 = \frac{\sum_{i=1}^{|D|} (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^{|D|} (x_i - \bar{x})^2} \quad w_0 = \bar{y} - w_1 \bar{x}$$

where $\bar{x}$ is the mean value of $x_1, x_2, \dots, x_{|D|}$, and $\bar{y}$ is the mean value of $y_1, y_2, \dots, y_{|D|}$. The coefficients w_0 and w_1 often provide good approximations to otherwise complicated regression equations.

Example

Table 5.1 shows a set of paired data where x is the number of years of work experience of a college graduate and y is the corresponding salary of the graduate. The 2-D data can be graphed on a scatter plot, as in Fig. 5.5. The plot suggests a linear relationship between the two variables, x and y . We model the relationship that salary may be related to the number of years of work experience with the equation $y = w_0 + w_1 x$.

x years experience	y salary (in 1000s)
3	30
8	57
9	64
13	72
3	36
6	43
11	59
21	90
1	20
16	83

Table 5.1 Salary Data

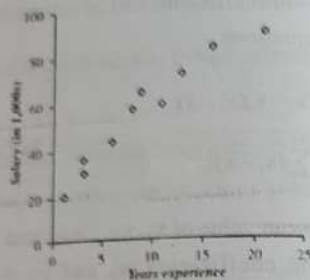


Fig. 5.5 2D Scatter Plot

Given the above data, we compute mean of $x = 9.1$ and mean of $y = 55.4$. Substituting these values into above equations, we get

$$w_1 = \frac{(3-9.1)(30-55.4) + (8-9.1)(57-55.4) + \dots + (16-9.1)(83-55.4)}{(3-9.1)^2 + (8-9.1)^2 + \dots + (16-9.1)^2} = 3.5$$

$$w_0 = 55.4 - (3.5)(9.1) = 23.6$$

Thus, the equation of the least squares line is estimated by $y = 23.6 + 3.5x$. Using this equation, we can predict that the salary of a college graduate with, say, 10 years of experience is ₹ 58,600.

Multiple Linear Regression

Multiple linear regression is an extension of straight-line regression so as to involve more than one predictor variable. It allows response variable y to be modeled as a linear function of n predictor variables or attributes, $A_1, A_2, \dots, A_n$, describing a tuple X . (That is, $X = (x_1, x_2, \dots, x_n)$.) Our training data set D , contains data of the form $(x_1, y_1), (x_2, y_2), \dots, (x_{|D|}, y_{|D|})$, where the x_i are the n -dimensional training tuples with associated class labels, y_i . An example of a multiple linear regression model based on two predictor attributes or variables A_1 and A_2 is

$$y = w_0 + w_1 x_1 + w_2 x_2$$

where x_1 and x_2 are the values of attributes A_1 and A_2 respectively in x . The method of least squares shown above can be extended to solve for w_0, w_1 and w_2 .

Non-linear Regression

Many nonlinear functions can be transformed into the above equation. Some nonlinear models can be modeled by a polynomial function. A polynomial regression model can be transformed into linear regression model. For example, $y = w_0 + w_1 x + w_2 x^2 + w_3 x^3$ can be converted to linear regression with new variables like $x^2 = x_2$ and $x^3 = x_3$. Thus the equation becomes a linear one as follows which can be solved using method of least squares discussed in linear regression.

$$y = w_0 + w_1 x + w_2 x_2 + w_3 x_3$$

5.9 Review Questions

1. What is classification?
2. Differentiate classification and prediction.
3. Differentiate supervised and unsupervised learning.
4. Explain issues in classification and prediction.
5. What is a decision tree?
6. Explain classification by decision tree induction.
7. What is information gain?
8. Explain Naïve Bayesian classification.
9. Explain rule based classification.
10. How will you extract rule from a decision tree?
11. What do you mean by the term coverage in classification?
12. What do you mean by accuracy in classification?
13. What are support vectors?
14. Explain the concept of support vector machines in classification.
15. What is a hyperplane?
16. Explain KNN algorithm.
17. What do you mean by learning from your neighbours?
18. Differentiate eager learning from lazy learning.
19. What is regression?
20. Explain linear regression with an example.
21. Discuss different types of regression.